An Experimental End-To-End GNN Explanation Framework

Emil Marcus Buchberg, Frederik Hecter Kowalski, Kent Vugs Nielsen

***Abstract*—In recent years, explainable AI (XAI) has gained a significant momentum, reflecting a necessity of better understanding AI decision-making. In domains like biomedicine, where the predictions are often of equally importance as the interpretability, the XAI field aids the advancement of research and development alike.**

Our paper introduces an innovative framework, that highlights the importance of explainability with Graph Neural Networks (GNNs) in graph classification downstream tasks. We have a specific focus on biomedical applications and by implementing the explainability method Class Activation Mapping (CAM), our goal is to showcase the interpretability of GNN outputs, using the graph-structured MUTAG dataset for our experiments. Our study includes the optimization of a Graph Convolutional Network (GCN) hyperparameters, to amplify the model's predictive power, by utilizing the state-of-the-art *Optuna* optimization framework. The GCN framework's effectiveness is quantified using explainability measures, such as Fidelity, Sparsity, and Contrastivity, followed by an analysis of important subgraphs identified in the study. Our research contributes to the sprouting field of GNN explainability, providing insights and methods that can be applied in future studies.

I. Introduction

MANY real-world phenomena can be viewed and modeled as graphs. One of the graph formalization powers lies in its ability to capture relationships, and not just individual properties of an object. Additionally, the graph formalism is flexible, providing a framework applicable to a diverse set of domains such as social networks, recommendation systems, chemical interactions and many others.

Analyzing networks has proven its merits in both transductive and inductive learning cases. Among the many use cases in graph machine learning, some of the most prevalent ones are node classification, link prediction, and graph classification. The center of our work revolves around graph classification, where a graph serves as a distinct data point, accompanied by its own label. Notably, graph classification differs inherently from tasks like node classification and link prediction as it follows an inductive approach. In this paradigm, a model is trained on labeled training examples to infer general rules which then are applied to test examples. Equivalent to traditional supervised learning, datasets are partitioned into separate training and test sets.

In contemporary research, deep learning models have surfaced as an instrumental tool for conquering downstream tasks in graph machine learning, including graph classification. Nevertheless, the straightforward application of pre-existing deep learning models proves to be unfeasible. For instance, convolutional neural networks are designed for grid-structured data like images, and transformer networks are optimized for sequential data like text [1]. Euclidean data differs from graph data for two reasons. First, graphs have a varying size and structure. Secondly, nodes in graphs do not have a natural ordering. Inherent differences to image or text data where the pixel or word order carries significant importance. Recognizing this gap, a new family of deep neural networks named graph neural networks (GNNs) has been developed. GNNs addresses the previously listed compatibility issues by being node order equivariant such that the final representations do not depend on node ordering as well as supporting graph inputs of varying size.

GNNs excel at leveraging the inherent structure of graphs by combining the topological information encoded in the graph's edges and nodes with the feature information associated with each node or edge. At the core of GNNs, one find the concept of message passing. This also applies to graph convolutional networks (GCNs) which we seek to utilize in this work. Message passing implements the notion of neighborhood information aggregation, allowing nodes to iteratively aggregate information from their neighbors and hence; capturing both structural and feature aspects.

While GCNs and GNNs in general are powerful in harnessing the rich information within graphs, their intricate internal workings are not amenable to interpretability, making it complex to interpret why and how specific predictions are made. This complexity often obscures the understanding of the specific reasons and mechanisms behind their predictions. In response to this challenge, numerous novel explainability methods have been developed, employing post hoc techniques to shed light on these processes. One such method, CAM, approaches explainability by mapping node embeddings from the final layer to the input space, thereby highlighting influential nodes [2]. It is important to recognize that these methods, including CAM, have their limitations. CAM operates under the assumption that the embeddings in the final convolutional layer accurately represent the significance of input features, an assumption that may not always hold true. To critically assess and quantify the reliability and effectiveness of these explanations, we employ three specialized metrics: Fidelity, Sparsity, and Contrastivity.

We seek to combine the presented methods in an end-to-end framework. The MUTAG dataset [3] suffices to demonstrate a GCN's ability to output predictions in a graph classification downstream task. A graph in MUTAG represents a nitro-aromatic compound and the task is to predict mutagenicity on Salmonella typhimurium [3]. In order to obtain the best possible performance, the GCN is subjected to a comprehensive hyperparameter optimization process. Moreover,

we adopt the CAM explainability method in an advance to understand our GCN's predictions. Hereby we discover the class specific atoms and substructures important in segregating nitro-aromatic compounds as either mutagen or non-mutagen. Finally, we evaluate the explanations using four metrics developed for explainability methods.

II. Related work

Application field: In the biomedicine industry, GCNs has already established themselves as a critical tool. They successfully address the unique challenges presented by graph-structured data, a natural way to represent molecules and biological reactions. Public benchmark graph datasets, such as the MUTAG dataset, provides the fundaments for developing and testing the GNNs ever-evolving architecture and techniques, and are crucial for further research applications. Utilizing GCNs in the field of biomedicine, provides a unique opportunity, to scope the interplay between the molecular structure and biological activity, and extract further insights [4, 5, 6]. The application of GCNs in biomedicine, not only aids the understanding of molecular behavior, but also contributes to advancing paramount areas like drug discovery and the like.

Explainability and interpretability: Many deep-learning models have been treated as black boxes with input-output behavior, and challenges emerges when attempting to interpret the reasoning of a model's decisions and pattern-recognition. Thus, the explainable AI (XAI) field has gained a significant momentum, signaling the requirements of trustworthy traceability, that allows for transparent insights in AI decision-making [7, 8]. Many explainability methods developed in the XAI field, does not account for complex data structures akin to graph data, and such, developing and integrating techniques for graph model explainability and interpretation has become an important sprout in the growing field of XAI. In recent years, many has drawn inspiration from the field of XAI when integrating and generalizing interpretability and explainability into the graph domain [2, 6]. CAM, being one such method, originates from the field of computer vision [9]. CAM utilizes the saliency of trained feature maps, linearly weighted with class weights obtained from the model, to distinguish the importance of discriminative features. By using CAM, it is possible to map the weighted feature maps back to the input image, and visualize the class-specific saliency. It has been shown that CAM, among other methods, can be applied directly to the architecture of GCNs, as the technique is reliant on feature maps, a design which is stemming from computer-vision domain, and can be closely translated into the graph domain. The adaptation of explenation methods [2, 6, 10] into the domain of GCNs, provides deeper insights into the model's decision-making process. It is a crucial step in making the models' predictions more transparent, especially in domains like biomedicine, where understanding the interpretability is just as important as the predictions themselves. Our work aligns with this philosophy, employing CAM to explain the predictions of our GCN on the MUTAG dataset. This approach allows us to pinpoint certain features and structures in the chemical compounds that contributes to mutagenicity. Thereby, we do not only rely blindly on predictions made by the model, but utilize the opportunity to interpret the biological effect, and identify key components that can help further the understanding of such downstream tasks.

Explainability measures: Additionally to incorporating explainability, efforts has been made to quantify the performance the explainability-methods provide of the models. We employ several of these measures in our work, mentioned by Yuan et al. [2], including *Fidelity*, *Sparsity* and *Contrastivity*. These quantitative metrics provides a framework for assessing the quality of explanations, generated by the GCN, and reliably ensuring their applicability. The study by Pope et al. [6] furthermore provides a comprehensive framework for assessing GCN explainability, and further includes an analysis of subgraphs, obtained by the explanation techniques, to identify recurring patterns in molecular graphs. By leveraging the growing sophistication in explainability research, Pope et al. are able to showcase interpretation beyond basic prediction, to more nuanced and detailed explainability and interpretability, that aims to solve the challenges posed by graph data.

III. Methods

We delineate the foundational principles and the philosophical framework underlying the implementation of hyperparameter optimization in Graph Convolutional Networks. Furthermore, we cover the essential theory for CAM applied to pinpoint significant features.

A. Bayesian Optimization

In order to maximize the performance of a GCN, one can perform hyperparameter optimization of the hyperparameters. We find the hyperparameters x that minimizes the objective function $f(x)$

$$x^* = \arg\min_{x \in \mathcal{X}} f(x). \tag{1}$$

Here, $f(x)$ is an error obtained in the evaluation of the validation set. Thus, the hyperparameters yielding the best score on the validation set metric are selected as the final hyperparameters.

Sequential Model-based Global Optimization (SMBO) algorithms are a formalization of Bayesian optimization under which numerous variants exists. These algorithms optimize hyperparameters by conducting successive trials, as illustrated in the generic SMBO in algorithm 1.

Algorithm 1 Algorithm of generic SMBO [11]

SMBO(f, M_0, T, S)
$\mathcal{H} \leftarrow X$
for $t \leftarrow 1$ to T **do**
  $x^* \leftarrow \text{argmin}_x S(x, M_{t-1})$
  Evaluate $f(x^*)$ ▷ Expensive step
  $\mathcal{H} \leftarrow \mathcal{H} \cup (x^*, f(x^*))$
  Fit a new model M_t to $\mathcal{H}$
end for
return $\mathcal{H}$

SMBO methods approximate f with a surrogate function S, a probability model of the objective function learned based

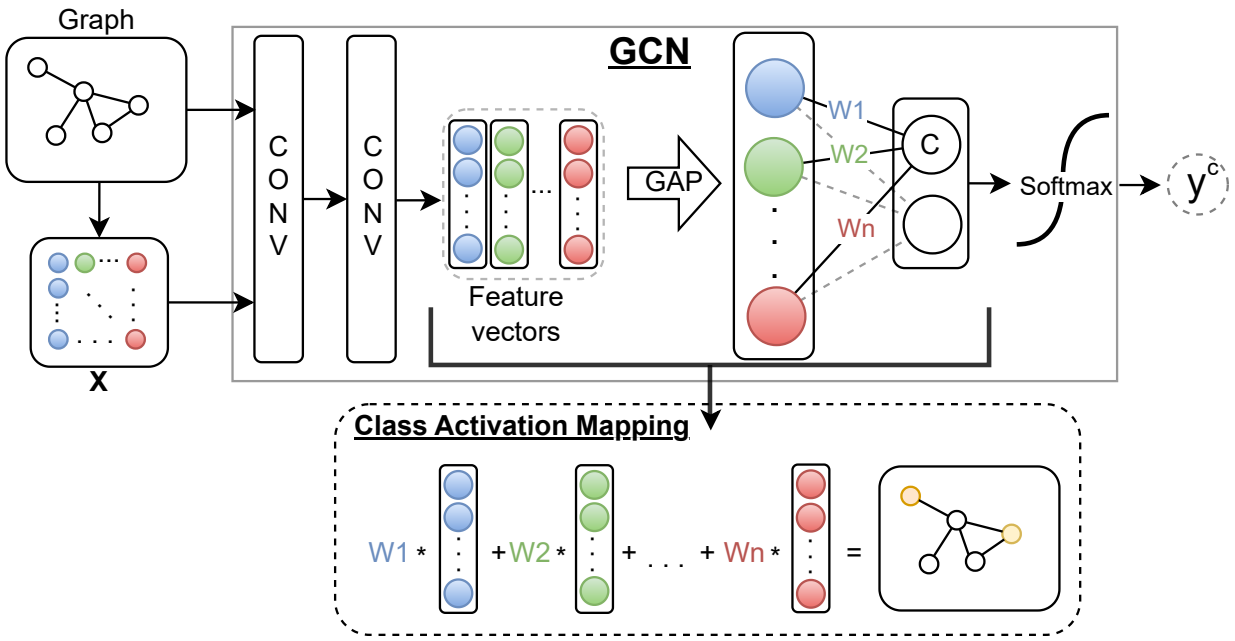


Fig. 1: Implemented GCN architecture, with CAM.

on previous evaluations. Rather than optimizing the objective function directly since it is more costly to evaluate, the point x^* that maximizes the surrogate is chosen as the next candidate for where the true objective function f should be evaluated. In order to select the best candidate for the surrogate model, a selection criterion like Expected Improvement [11] is used. In each iteration, $\mathcal{H}$ is appended with the pair $(x^*, f(x^*))$, and subsequently used to construct a refined surrogate model. Given that the surrogate model accurately represent the objective function, intuitively it should yield the true optimal hyperparameter values under the selection criterion. SMBO methods differ in how they build a surrogate of the objective function and their selection of S, however we refrain from delineating the details of TPE.

B. Applied GNN Framework

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph $\mathcal{G}$, with its set of vertices $\mathcal{V}$ and set of edges $\mathcal{E}$. The adjacency matrix is indexed by the nodes in the graph and denoted by $A \in \mathbb{R}^{N\times N}$ where N represents the total number of nodes in the graph $\mathcal{G}$. The MUTAG dataset includes node attributes, and the attribute matrix is denoted as $X \in \mathbb{R}^{N\times F}$, where F is the number of node attributes. The general notion of GNN's is to learn node representations by effectively embedding node neighborhoods from $\mathcal{G}$. To do so, we aggregate over every node neighborhood, and combine it with earlier embeddings as node representations $H = \mathbb{R}^{N\times C}$ [12], where C is the number of channels. The first embedding is the feature matrix itself, and the number of channels can be a layerwise adjustable hyperparameter. We can consider it as a l-layerwise algorithm:

where the nodes have their final representation at H^L, and $N(i)$ is the neighborhood of node j.

Algorithm 2 GNN framework[12]

$H^0 = X$ ▷ Initialize the feature matrix
for $l = 1, \cdots, L$ **do**
  $a_i^l = \textbf{AGGREGATE}^l\{H_j^{l-1} : j \in N(i)\}$
  $H_i^l = \textbf{COMBINE}^l\{H_i^{l-1}, a_i^l\}$
end for

C. Extension to GCN

When we consider algorithm 2 in the context of neural network architecture, a propagation rule is required to efficiently calculate the embeddings through the network. The propagation rule [13]

$$H^{l+1} = \sigma(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^l W^l), \tag{2}$$

proposed by Kipf & Welling [13] is a powerful and localized filter, that utilize the general concepts of algorithm 2. Firstly in eq. (2), we utilize that $\tilde{A} = A + \mathbf{I}$, where $\mathbf{I}$ is the identity matrix, to build node self-connection. This allows for a node's features to be included when computing the layerwise propagation. We further define $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$, which is utilized to symmetrically normalize $\tilde{A}$. W^l is a layerwise learnable weight matrix, that multiplied with H^l and activated with an activation function σ, projects layerwise feature vectors. When considering eq. (2) on a node-level, we can rewrite the equation to:

$$H_i^l = \sigma\left(\sum_{j\in\{N(i)\cup i\}} \frac{\tilde{A}_{ij}}{\sqrt{\tilde{D}_{ii}\tilde{D}_{jj}}} H_j^{l-1} W^l\right). \tag{3}$$

We notice the sum in eq. (3) aggregate over a node's neighborhood $N(i)$, and combine it, like presented in algorithm 2.

We can re-write the **COMBINATION** function in the sum by considering the properties of the given node [12]:

$$H_i^l = \sigma\left(\sum_{j\in N(i)} \frac{A_{ij}}{\sqrt{\tilde{D}_{ii}\tilde{D}_{jj}}} H_j^{l-1}W^l + \frac{1}{\tilde{D}_i}H_i^{l-1}W^l\right).$$

It is clear how the neighbor-node j is normalized by the degrees of the two nodes, i, j, given the self-connected adjacency matrix $\tilde{A}$. By using the **COMBINE** function, we aggregate over all of i's neighbors and itself.

To extend the node classification to the downstream task of graph classification, it is necessary to add a global pooling layer after the final convolutional layer. There are several applications and variants of global pooling, however we utilize the global average pooling (GAP) [14]. By letting $h_k \in \mathbb{R}^{N\times C}$ be a feature map resulting from a convolutional layer, GAP can be expressed by

$$\text{GAP}(H_k^l) = \frac{1}{N}\sum_i h_{k,i}.$$

GAP can be interpreted as the k^{th} feature mean of H from the l^{th} convolutional layer, that is, the mean of H's individual column vector(s). This is also known as the readout layer, observed on fig. 1. Once a global pooling layer has been applied, it is possible to pass it through an activation function that maps to a final prediction on a graph level.

D. CAM

The class weights obtained from a GCN (similar to $w_1, \cdots w_n$ in fig. 1) are utilized to identify discriminative inputs of the GCN that drastically impacts output responses of the network. To do so, we compute a weighted sum for individual nodes, by utilizing the features, output by the last convolution layer, with the trained class weights of the network. By using the CAM technique, the network architecture is limited to a particular design. In the scenario of classification with softmax, the activations following the last convolutional layer should be fed directly into a GAP, then followed by a dense layer, where each class is individually represented by a neuron. The weights, connecting to the predicted class in the dense layer, can then be used to weight the importance of the features, and thus the node importance.

Formally, for a given input graph, let f_k be the resulting activations from the k^{th} feature, obtained from the last convolutional layer in the network. By applying GAP to the activations of f_k, we obtain

$$\text{GAP}(f_k) = e_k,$$

and thus, for a given class c the input for the softmax activation function S^c becomes

$$S^c = \sum_k w_k^c e_k, \tag{4}$$

where w_k^c are the learnable weights in the dense layer from the k^{th} GAP belonging to class c. The winning class is the highest y^c:

$$y^c = \frac{exp(S^c)}{\sum_c exp(S^c)}.$$

Zhou et al.[9], states, that by evaluating e_k at a single point (that is, a node), is equivalent to $\sum_i f_{k,i}$. By plugging $e_k = \sum_i f_{k,i}$ into eq. (4), we obtain

$$\begin{aligned} S^c &= \sum_k w_k^c \sum_i f_{k,i} \\ &= \sum_i \sum_k w_k^c f_{k,i}, \end{aligned}$$

and we define the nodewise class activation map as

$$\text{M}_i^c = \sum_k w_k^c f_{k,i}. \tag{5}$$

Therefore, $S^c = \sum_i M_i^c$, and the nodewise CAM, M_i^c, must be a direct indicator for how important a node has been in the predicting outcome of the winning class c.

IV. Materials

We present a comprehensive examination of the dataset employed in the conducted experiments, clarifying the configuration employed in the execution of training, testing, validation, and explainability procedures.

A. Dataset

The development of the MUTAG dataset was driven by the imperative to address the protracted exposure to drugs and their metabolic byproducts, with a specific focus on long-term toxic implications. A timely identification of such potential toxicity during the early stages of drug development holds significant promise in mitigating unwarranted financial expenditures and time losses [3]. A comprehensive review and analysis of extant literature, yielding insights into over 200 chemical compounds characterized by the presence of nitro groups (NO2) and exhibiting aromatic or heteroaromatic properties, undertook by Debnath et al. [3]. The Ames test, a mutagenicity assessment, was executed utilizing the bacterial strain Salmonella typhimurium TA98, renowned for its susceptibility to a diverse array of mutagenic agents. Among the aforementioned 200 chemical compounds, a quantitative structure-activity relationship has been formulated for 188 congeners. In particular, MUTAG is a collection of nitro-aromatic compounds divided into two classes according to their mutagenic effect on a bacterium. The goal is to predict their mutagenicity on Salmonella typhimurium. Graphs are used to represent chemical compounds, where vertices stand for atoms and are labeled by the atom type (represented by one-hot encoding), while edges between vertices represent bonds between the corresponding atoms. Consequently, the MUTAG dataset includes 188 graphs with 7 discrete node labels [15]. The various node labels are given as Carbon (C), Nitrogen (N), Oxygen (O), Fluorine (F), Iodine (I), Chlorine (Cl), and Bromine (Br). The edge labels are given as aromatic, single, double, and triple. However, we do not utilize the edge labels in our GCN. The dataset prompts the undertaking of graph classification tasks, specifically involving the prediction of the mutagenic status of a given graph. The distribution of

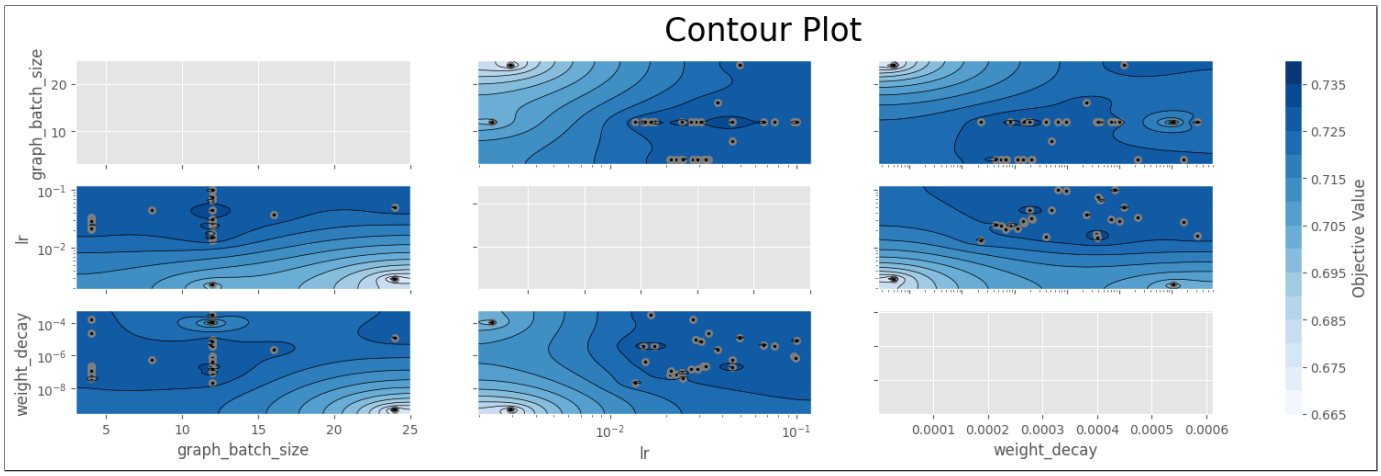


Fig. 2: Hyperparameter optimization contour plot.

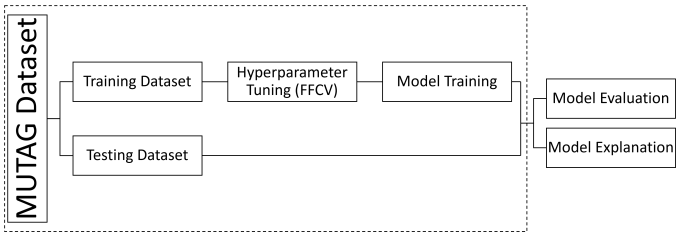


Fig. 3: Flowchart of data-flow

the graph labels are given as 125 for the positive class (1) and 63 for the negative class (0).

Hydrophobicity emerges as a prominent determinant influencing the mutagenic efficacy of aromatic nitro compounds. Remarkably, this aspect has been largely disregarded in existing literature. Furthermore, the heightened mutagenicity observed in electron-attracting elements conjugated with nitro groups underscores a noteworthy association. Additionally, compounds characterized by the presence of three or more fused rings exhibit markedly higher mutagenic activity, all other factors held constant, in comparison to those possessing one or two fused rings [3]. Although aromatic nitro compounds have been identified as exhibiting mutagenic properties, and the entirety of the dataset is characterized by the presence of nitro groups, the absence of a definitive ground truth for ascertaining mutagenicity necessitates reliance on certain characteristics. These characteristics provide compelling reasons to hypothesize the presence or absence of such mutagenic attributes. This highlights the essential requirement for robust domain knowledge when assessing mutagenic attributes.

B. Training, Validation & Testing

The dataset has been split into two subsets: one designated for the purpose of training the model and fine-tuning hyperparameters, while the other serves as a holdout set for assessing the performance of the model. Additionally, the latter subset facilitates the quantification of the model's explanatory capacity with respect to its predictions. Specifically, the 188 graphs has been partitioned utilizing an 80/20 percent stratified train-test split, and the data-flow is illustrated in fig. 3. Consequently, the training set consists of 150 graphs, while the test set includes the remaining 38 graphs. Stratified Five-fold cross-validation (SFFCV) is employed to ascertain the optimal hyperparameters for the GCN model. The entirety of the training set is subject to SFFCV, a necessary approach due to the limited number of graphs within the dataset. Subsequently, upon determination of the optimal hyperparameters, the GCN model undergoes training on the 150 graphs constituting the training set, configured to employ the identified hyperparameters. The final hyperparameters are defined in table I. The GCN model's performance is evaluated using the test set and its explainability is evaluated and is explained in detail in section III and section VI.

V. GCN Implementation & Results

In this section, we present the hyperparameter optimization results, obtained from rigorous training and testing. Furthermore, we showcase the graph classification accuracy of our GCN on the MUTAG dataset.

A. Hyperparameter Tuning

We estimate the optimal values for our network's learning rate, weight decay and graph batch size.

1) *Learning rate*: dictates the step size the optimizer takes during the weight updating process.
2) *Weight decay*: a regularization technique to mitigate overfitting.
3) *Graph batch size*: defines the number of graph samples propagated through the model before updating its parameters.

Hyperparameter optimization was done using the Optuna framework, a library that facilitates and simplifies implementation of hyperparameter tuning concepts [16]. Notably, we employed TPE using Optuna. We list the configurations tested for each hyperparameter in table I. After 200 trials, we achieved our best results, with trial 72 reaching an accuracy of 73.62% using the hyperparameter values listed in table I.

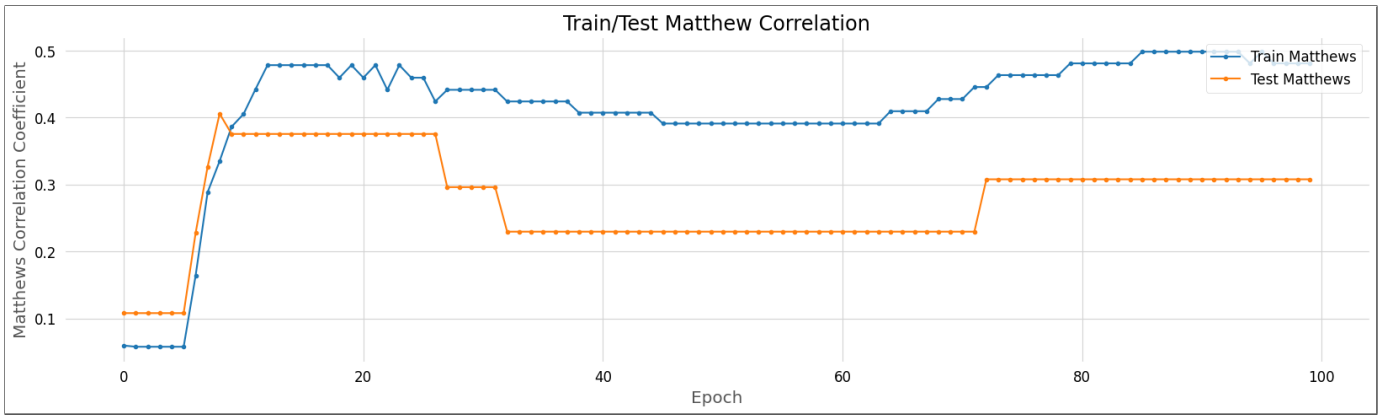


Fig. 4: Training/test MCC over a 100-epoch training period.

TABLE I: Hyperparameter configuration options and results.

Hyperparameter	Configurations	Result
Learning rate	$[1e-5, 1e-1]$	**0.01512**
Weight decay	$[1e-10, 1e-6]$	**3.8317e-06**
Graph batch size	4, 8, 12, 16, 24, 32	**12**

Figure 2 reveals more about the TPE sampler behavior and the relationships among the hyperparameters. The plot demonstrates the Bayesian approach of the TPE sampler as only very few points are located in light colored regions and more points are centered around the identified local minimums. This provides an idea of the hyperparameter search-space Optuna is considering. Other observations include that graph batch sizes bigger than 12 are excluded by the TPE, learning rates higher than 10^{-2} are seen to yield higher accuracy, and the optimal weight decay is largely dependent on the learning rate.

B. Graph Classification Results

We present the classification results obtained on the MUTAG. A critical part of our explainability analysis involves interpreting these results to evaluate whether the model successfully learned a meaningful representation of the dataset. Notably, if the model fails in this fundamental aspect, the associated explainability analysis would like yield ambiguous results.

The model was implemented with PyTorch and Pytorch Geometric [17, 18]. Our model conforms to the GCN architecture outlined in section III-C. This architecture comprises two convolutional layers, each with a channel size of 7, a GAP layer, and concludes with a dense layer with seven neurons leading to two softmax output neurons. As described in section IV-B, for training and evaluation purposes we employed an 80-20 stratified split of the dataset, ensuring a balanced class representation in both the training and test sets. The predictions made on the test set comprises the final result.

In assessment of the model performance, we employ a comprehensive suite of evaluation metrics, each contributing insights into the model's effectiveness and reliability from different perspectives. The presence of a majority class provides grounds to assess the performance of the model from different viewpoints rather than a simple accuracy score. We report the scores for the following metrics: Accuracy, Recall, Precision and Matthews Correlation Coefficient (MCC). MCC is a correlation coefficient between observed and predicted classification. It returns a value between -1 and 1; -1 indicating total disagreement, 0 equivalent to random prediction and 1 to a perfect prediction.

$$\text{MCC} = \frac{TP * TN - FP * FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

where TP is the number of true positives, TN is the number of true negatives, FP is the number of false positives and FN is the number of false negative. MCC has the advantage that it only generates a high score if the model correctly predicts the majority of the positive instances and the majority of the negative instances [19]. Thus penalizing the model if it only correctly predicts the majority class.

To determine the optimal number of epochs, we propose an initial overhead of 100 epochs. As illustrated in fig. 4, which displays the MCC for both the training and testing set over a 100-epoch range, a notable decline in performance becomes apparent after around 30 epochs. This trend of stagnation persists from epoch 30, after which a resurgence in performance is seen from epoch 70 and onward. Initially, a 25-epoch range seems to suffice based on the test MCC. However, in the later stages of our research, the model exhibits traits of underfiting, thus we instead select a model trained over an 80-epoch training period.

The configuration produces the results seen in table II. Authors of [20] report an accuracy of 0.78 on MUTAG and uses a GCN with the same architecture, which is close to the one reported here at 0.7105. Thus, our model achieves comparable performances to others in the literature.

Unsurprisingly, the model achieves a high precision of 0.88. Given that majority of the observations are positive and a high precision implies our model correctly predicts the positive class instances, it is clear that model struggles to discern the negative class instances and exhibits a tendency to predict positive for all instances. The underlying numbers suggest the same, with 5 TN instances and 3 FN instances. Additionally, the model tends to predict positive even for half of the negative

TABLE II: Metric results for 80-epoch trained model.

Accuracy	Precision	Recall	MCC
0.7105	0.88	0.7333	0.3079

instances; seen by the 8 FP instances. This is clearly reflected in the relatively low MCC score at 0.3079.

VI. Explainability

A. Evaluation Metrics

The assessment of human evaluations significantly hinges on subjective interpretations and specialized domain knowledge, potentially leading to inequities in fairness. While visualization outcomes may offer valuable insights into the human-perceived reasonableness of explanations, the reliability of such evaluations is constrained by the absence of established ground truths [2]. Consequently, the implementation of robust evaluation metrics is imperative for the analysis of explanation methods. This section delineates various evaluation metrics specifically tailored for tasks involving explanations.

1) Fidelity: Fidelity, as a concept, pertains to the identification of salient features deemed important for a model's performance. This is achieved by quantifying the diminution in model accuracy following the masking of input features with attribution values surpassing a specified threshold. The underlying intuition is that if the input features, as identified by explanation methodologies, are indeed discriminative for the model, then their omission should result in substantial alterations in the model's predictions. Consequently, Fidelity is quantitatively delineated as the discrepancy in accuracy between the model's initial predictions and those subsequent to the masking of these important input features [2, 6].

In a formal context, Fidelity is split into two distinct metrics: Fidelity+ and Fidelity-. Fidelity+ examines the variation in prediction accuracy consequent to the exclusion of important input features. Conversely, Fidelity- focuses on the alteration in predictions following the removal of features deemed unimportant. The underlying rationale posits that important features, being repositories of discriminative information, should yield predictions similar to the original ones even in the absence of unimportant features. Hence, these two metrics are given as follows [2]

$$\textit{Fidelity+} = \frac{1}{N}\sum_{i=1}^{N}(\mathbb{I}(\hat{y}_i = y_i) - \mathbb{I}(\hat{y}_i^{m_i} = y_i)),$$

$$\textit{Fidelity-} = \frac{1}{N}\sum_{i=1}^{N}(\mathbb{I}(\hat{y}_i = y_i) - \mathbb{I}(\hat{y}_i^{1-m_i} = y_i)),$$

where N represents the total number of graphs within the dataset. The term y_i is the label attributed to the i^{th} graph. Conversely, $\hat{y}_i$ denotes the original prediction for the i^{th} graph. The notation $1 - m_i$ refers to the complementary mask, designed to omit unimportant features. As a result, $\hat{y}_i^{1-m_i}$ refers to the prediction yielded upon processing the complementary masked graph through the trained model.

2) Sparsity: Sparsity is a metric conceived to quantify the degree of localization within an explanation. This measure is of particular use in the context of analyzing extensive graphs, where a manual examination of all nodes is impractical [6]. Sparsity evaluates the proportion of features identified as important by explanation methods. The underlying assumption is that optimal explanations should exhibit a high degree of sparsity; conversely, explanations lacking sparsity suggest an over-reliance on a majority of the graph nodes, if not all. This leads to less effective explanations, especially in scenarios requiring to localize the most significant nodes.

Formally, the Sparsity metric can be computed as [2]

$$\textit{Sparsity} = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{|m_i|}{|M_i|}\right),$$

where $|m_i|$ denotes the number of important input nodes and $|M_i|$ is the total number of nodes in graph i.

3) Contrastivity: Contrastivity is formulated to calculate the normalized disparity of saliency maps corresponding to diverse classes, thereby providing class-specific explanations. This metric is developed to encapsulate the notion that features underscored by an explanation method as being class-specific ought to exhibit variance across different classes [6]. Consequently, Contrastivity is defined as the quotient of the Hamming Distance between binarized heat-maps for positive and negative classes. This quotient is then normalized by the total number of nodes by either of the two classes involved.

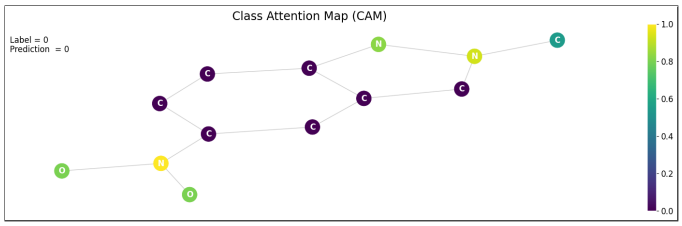
Formally, the Contrastivity metric can be computed as [6]

$$\textit{Contrastivity} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{d_H(m_i^0, m_i^1)}{|m_i^0| \vee |m_i^1|}\right),$$

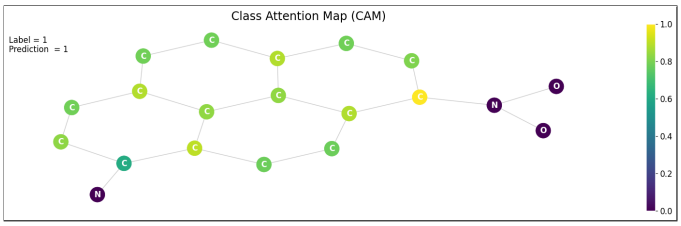
where, d_H is the Hamming Distance between two distinct classes, m_i^0 and m_i^1, corresponding to the i^{th} graph. Specifically, m_i^0 is defined as the masked graph for the negative class, wherein the weights associated with the negative class are utilized for identifying salient features. Conversely, m_i^1 is characterized similarly for the positive class, employing weights from the positive class for the determination of significant features.

B. Explainability Results

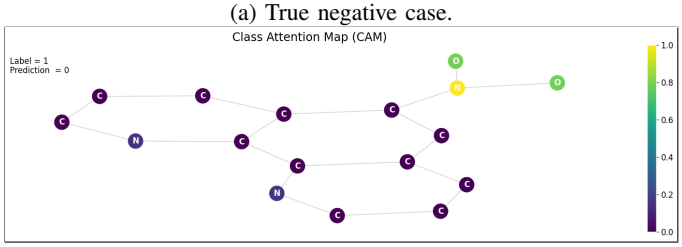
We now present the explanations obtained with CAM. These are the underlying explanations for the results in section V-B. Explanations are evaluated with *Fidelity+*, *Fidelity-*, *Sparsity* and *Contrastivity* that we defined in section VI-A. When constructing importance maps we adhere to Min-Max scaling, as recommended by Yuan et al. in [2]. Min-Max scaling promotes a proportionate view of node importance scores. We have also implemented a lower bound of 0, ensuring that all negative values are set to 0. This approach effectively eliminates the possibility of attributing importance to nodes with negative CAM values, maintaining the focus on positively contributing elements. Additionally, all metrics rely on a masking threshold between 0 and 1, deeming nodes as either important or unimportant. The masking is conducted on the normalized CAM values.



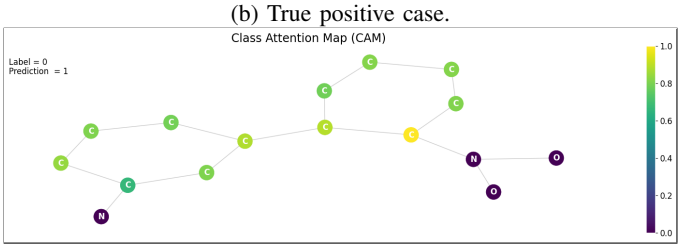
(a) True negative case.



(b) True positive case.



(c) False negative case.



(d) False positive case.

Fig. 5: Four explanation graphs from the test set. The color of each node corresponds to its relative importance within each graph with 1 being most important and 0 as least important.

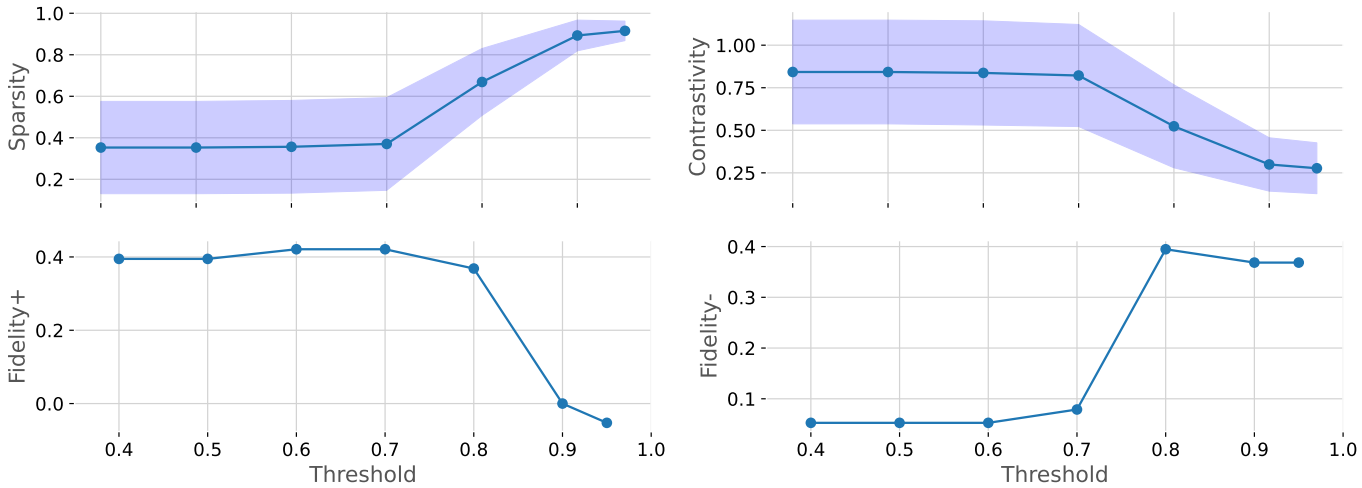


Fig. 6: Fidelity-, Fidelity+, Sparsity and Contrastivity results with different masking thresholds. Sparsity and contrastivity include standard deviations.

First, we present two explanation graphs for either class. fig. 5a shows predominant importance scores assigned to the nitro group and to nitrogen atoms. These importance scores lead to a correct prediction of a negative class instance. In contrast, fig. 5c illustrates similar importance scores assigned to the N02 group and zero to none importance assigned to the remaining atoms, leading to an incorrect prediction of a positive class instance. Thus, presence of single N02 group is no evidence for either class, and the model potentially disregarded stronger evidence of the positive class. This argument aligns with the importance scores in fig. 5b, showing high importance scores for the carbon fused rings, similar to the ones in fig. 5c, and zero importance scores for the NO2 group. However, the discrepancy lies within the chemical rings that exclusively comprises carbon atoms in fig. 5b but feature nitrogen atoms in fig. 5c. One could postulate that this discrepancy disrupt the model from interpreting these as evidence for the positive class. Similarly in fig. 5d, the model assigned carbon rings high importance, albeit non-fused rings, and thus leading the model to incorrectly predict the graph as a positive class instance. Whether these findings generalizes to the other graphs in the test set remains to be studied in section VI-C.

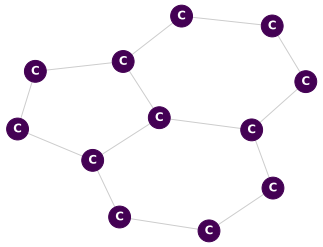
We now iterate the final evaluation results seen in fig. 6. Again, the results are highly dependent on the chosen masking threshold, hence why we show results for multiple threshold values. Figure 6 delineates the relationship between four critical evaluation metrics and seven varying thresholds. We observe a consistent sparsity rate for thresholds ranging from 0.4 to 0.7, with sparsity scores at 0.4 and standard deviations around 0.2. Such a standard deviation implies a degree of variability in graph sizes or the count of important nodes. Notably, a direct correlation exists between sparsity and stricter masking thresholds exceeding 0.7.

Optimal explanations are sparse and show clear class distinctions; higher contrastivity are evidence of this. The pro-

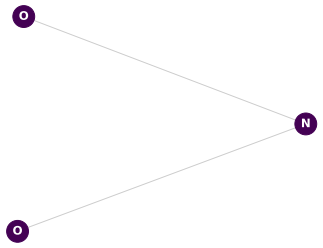
nounced contrastivity underscores a distinct differentiation in the importance of nodes between winner and loser classes as values lie in the 0.8-0.85 range. This measure of contrastivity, however, declines when the threshold exceeds 0.7, indicating an excessively stringent masking threshold.

Analyzing our fidelity+ scores, it becomes apparent that masking nodes with an importance exceeding 0.8 leads to the removal of numerous key nodes in a graph, as reflected by a significant drop in the fidelity+ score. This trend suggests that nodes with importance scores near 0.8, as assessed by CAM, are crucial for the accuracy of our model's predictions. The findings for fidelity- mirror this observation. Retaining nodes with an importance score above 0.7, while masking those below this threshold, results in negligible accuracy disparities. This indicates that essential features are effectively masked at thresholds above 0.7, thereby impacting model performance.

All things being equal, the CAM methodology highlights salient features within the model, as evidenced by all four metrics. Most notably, a significant shift in accuracy is observed when masking nodes deemed important by CAM. Conversely, preserving these important nodes does not lead to a notable shift in accuracy, underscoring the efficacy of the CAM approach in identifying critical model features.



(a) Positive Class Prediction



(b) Negative Class Prediction

Fig. 7: Frequent identified substructures for the positive and negative class prediction.

C. Subgraph Frequency Analysis

As discussed in section IV-A of this study, certain elements have been identified as potential enhancers of mutagenic activity. It is particularly significant to note that compounds containing a structure of three or more fused rings demonstrate a substantially elevated level of mutagenic activity, assuming other variables remain constant, in comparison to compounds with only one or two fused rings. Moreover, while aromatic nitro compounds have been acknowledged for their mutagenic properties, this does not unequivocally guarantee their uniform manifestation as mutagens in all instances. In our research, we undertake a systematic examination of prominent substructures within the dataset, focusing on identifying recurrent patterns. To achieve this, we quantify the frequency of each identified salient substructure within the test dataset with regards to its predicted label using a masking threshold of 0.7. This methodical approach allows for a more nuanced understanding of the data, particularly in terms of the predictability and consistency of these salient substructures.

As presented in fig. 7a, the analysis reveals that the predominant substructure associated with positive class predictions is a ring-like structure. Specifically, there are eight out of twenty-nine graphs designated as positive class predictions exhibit a structure comprising three rings, exclusively composed of carbon atoms. Additionally, a notable proportion (17 out of 29) of these graphs feature three or more fused rings, albeit not limited solely to carbon atoms. This observation aligns with the theoretical expectations, particularly considering the presence of chemical compounds characterized by three fused rings, a structural feature strongly indicative of mutagenic activity.

In contrast, fig. 7b illustrates that frequent substructure corresponding to negative class predictions is the aromatic nitro compound. Specifically, it is observed that all eight remaining graphs classified under the negative prediction category contain one or more aromatic nitro compounds. This outcome is consistent with expectations, given that aromatic nitro compounds are ubiquitous across all graphs in the dataset. This also emphasizes that the presence of aromatic nitro compounds does not conclusively confirm their role as mutagens. Thus, the model assigns significant weight to these nodes in predicting the negative class.

VII. Conclusion

In section III-C, we assess the theoretical foundations of GCNs. This section elucidates the key principles that enable GCNs to effectively process graph-structured data. We showcase the proficiency of a GCN in a graph classification task, particularly in predicting the mutagenicity of bacteria. The outcomes of this task are detailed in section V-B. While the results indicate a moderate success in distinguishing between positive and negative class instances, the model's limited ability in identifying negative class instances and its propensity to mistakenly classify negative instances as positive, likely caused by a preponderance of positive instances, significantly impairs its performance.

Section III-C introduces the established explanation method, CAM. Utilizing CAM, we endeavor to elucidate the results from the classification task by pinpointing key features influential in determining the winner-class. These insights are further quantified using four evaluation metrics: Fidelity+, Fidelity-, Sparsity, and Contrastivity, discussed in section VI. The fidelity+ metric reveals a notable 0.4 shift in accuracy upon masking critical features. In contrast, the fidelity- score indicates minimal to no change in accuracy when preserving important features. Additionally, a contrastivity value of

0.8 highlights the distinct separation of feature importance between winner and loser classes. As a final analytical step, we examine prevalent substructures derived from the feature explanations. In section VI-C, our analysis identifies ring structures as commonly associated with positive class predictions and NO2 groups as frequently occurring in negative class predictions. These findings are in line with the known characteristics of each class.

VIII. Future work

For our future work in the graph domain, we intend to further investigate the interpretability and explainability of GNNs within the biomedicine field. This will involve the integration of Gradient-weighted Class Activation Mapping (Grad-CAM), an extension of CAM that relaxes the constraints, potentially providing a deeper and better understanding of our GCN's predictions. Additionally, we want to delve into more advanced explainability methods, particularly focusing on implementing more advanced innovations such as the GNNExplainer [21], a method highlighted by Yuan et al. in [2]. Alongside these particular methods, we have a goal to expand and apply our knowledge of XAI techniques to appropriate graph-structured data, thereby improving our model's transparency and understandability in biomedicine. This expansion will cater to improving the clarity and interpretability of our models, and also open our horizon for applying GNNs across various domains where graph-structured data plays a vital role. With these efforts, we aspire to achieve new benchmarks in the GNN and XAI research fields, particularly in terms of transparency, applicability, and real-world utility.

Ideally, we want to include additional datasets for the downstream task, graph classification. This integration is anticipated to yield a more comprehensive understanding of the performance capabilities of explanatory methods. Furthermore, such an approach facilitates the evaluation of the model's architecture across a spectrum of datasets. Empirical evidence suggests a correlation between the proficiency of a model and the quality of its explanations [10]. Consequently, the deployment of a variety of architectures and tailored models is likely to be a pivotal factor in optimizing both the model's performance and the quality of its explanations.

Acknowledgements

We leveraged the assistance of ChatGPT [22] and GitHub Copilot [23], which provided valuable code suggestions, expediting the development process and serving as a source of inspiration when encountering challenges. This study was made possible by Aalborg University (AAU). The dataset was freely available and provided by the PyTorch Geometric team. The work presented in this paper was supervised by Ehsan Mobaraki of Aalborg University.

References

[1] Benjamin Sanchez-Lengeling et al. "A Gentle Introduction to Graph Neural Networks". In: *Distill* (2021). https://distill.pub/2021/gnn-intro. DOI: 10.23915/distill.00033.

[2] Hao Yuan et al. "Explainability in graph neural networks: A taxonomic survey". In: *IEEE transactions on pattern analysis and machine intelligence* 45.5 (2022), pp. 5782–5799.

[3] Asim Kumar Debnath et al. "Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity". In: *Journal of Medicinal Chemistry* 34.2 (1991), pp. 786–797. DOI: 10.1021/jm00106a046. eprint: https://doi.org/10.1021/jm00106a046. URL: https://doi.org/10.1021/jm00106a046.

[4] Yu-Chen Lo et al. "Machine learning in chemoinformatics and drug discovery". In: *Drug Discovery Today* 23.8 (2018), pp. 1538–1546. ISSN: 1359-6446. DOI: https://doi.org/10.1016/j.drudis.2018.05.010. URL: https://www.sciencedirect.com/science/article/pii/S1359644617304695.

[5] Ryosuke Kojima et al. "kGCN: A graph-based deep learning framework for chemical structures". In: (Feb. 2020). DOI: 10.21203/rs.2.23904/v1.

[6] Phillip E. Pope et al. "Explainability Methods for Graph Convolutional Neural Networks". In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 10764–10773. DOI: 10.1109/CVPR.2019.01103.

[7] *IBM: What is explainable AI?* https://www.ibm.com/topics/explainable-ai. [Online; accessed 14-dec-2023].

[8] Prashant Gohel, Priyanka Singh, and Manoranjan Mohanty. *Explainable AI: current status and future directions*. 2021. arXiv: 2107.07045 [cs.LG].

[9] Bolei Zhou et al. *Learning Deep Features for Discriminative Localization*. 2015. arXiv: 1512.04150 [cs.CV].

[10] Ehsan Bonabi Mobaraki and Arijit Khan. "A Demonstration of Interpretability Methods for Graph Neural Networks". In: *Proceedings of the 6th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)* (2023). URL: https://api.semanticscholar.org/CorpusID:259213245.

[11] James Bergstra et al. "Algorithms for Hyper-Parameter Optimization". In: *Proceedings of the 24th International Conference on Neural Information Processing Systems*. NIPS'11. Granada, Spain: Curran Associates Inc., 2011, pp. 2546–2554. ISBN: 9781618395993.

[12] Lingfei Wu et al. *Graph Neural Networks: Foundations, Frontiers, and Applications*. Singapore: Springer Singapore, 2022, p. 725.

[13] Thomas N. Kipf and Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. 2017. arXiv: 1609.02907 [cs.LG].

[14] *Global mean pool*. https://pytorch-geometric.readthedocs.io/en/latest/modules/nn.html. [Online; accessed 10-dec-2023].

[15] Claudio Gallicchio and Alessio Micheli. "Fast and deep graph neural networks". In: *Proceedings of the AAAI*

conference on artificial intelligence. Vol. 34. 04. 2020, pp. 3898–3905.

[16] Takuya Akiba et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. 2019. arXiv: 1907.10902 [cs.LG].

[17] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. URL: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

[18] Matthias Fey and Jan E. Lenssen. "Fast Graph Representation Learning with PyTorch Geometric". In: *ICLR Workshop on Representation Learning on Graphs and Manifolds*. 2019.

[19] D Chicco and G Jurman. "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation". In: *BMC Genomics* 21.6 (2020). URL: https://doi.org/10.1186/s12864-019-6413-7.

[20] Yao Ma et al. "Graph Convolutional Networks with EigenPooling". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '19. Anchorage, AK, USA: ACM, 2019, pp. 723–731. ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330982. URL: http://doi.acm.org/10.1145/3292500.3330982.

[21] Rex Ying et al. *GNNExplainer: Generating Explanations for Graph Neural Networks*. 2019. arXiv: 1903.03894 [cs.LG].

[22] OpenAI ChatGPT (2022). *ChatGPT: Optimizing Language Models for Dialogue.* URL: https://openai.com/blog/chatgpt.

[23] Nat. Friedman. *Introducing GitHub Copilot: your AI pair programmer.* URL: https://copilot.github.com/.